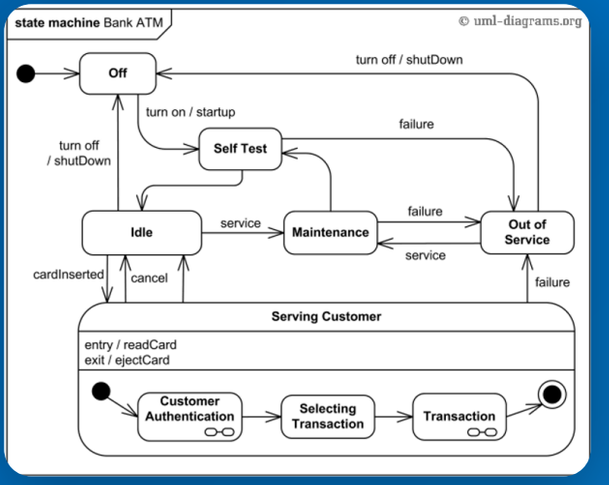
ניתוח ועיצוב מערכות מידע

מתודולוגיות פיתוח

הרצאה 11



עודכן: 07.11.2026

סדר היום

1. **למה בכלל צריך שיטה**
2. מפל מים ואב-טיפוס
3. תוספתי, איטרטיבי וספירלה
4. Agile ו-Scrum
5. בחירה, AI וסיכום

Artificial Intelligence = AI · בינה מלאכותית

מתודולוגיה אינה שלב — היא צורת הצינור

אותם שמונה שלבים שאתם מכירים. מה שמשתנה הוא באיזה גודל מנה ובאיזה סדר עוברים בהם

בסוף ההרצאה הקודמת המלצנו "להעלות בשלבים — קפיטריה אחת בסמסטר א'". זו כבר הייתה החלטה מתודולוגית, פשוט בלי לקרוא לה בשם. היום נותנים לה שם.

מעבר אחד ארוך

כל שלב פעם אחת, לכל המערכת



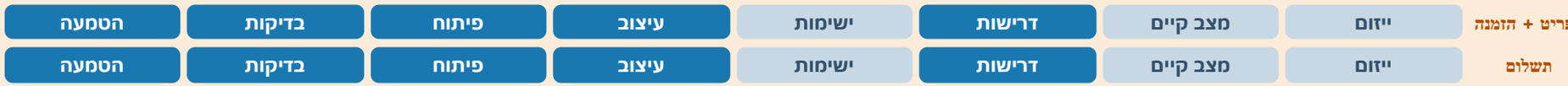
מעברים חוזרים

כל סבב עובר בכל השלבים — ומשפר את אותה מערכת



פרוסות

כל פרוסה היא חלק עובד מהמערכת, מקצה לקצה



כל מה שלמדתם עד היום הוא שלב. היום לומדים מה מחליט על הסדר, הגודל והחזרתיות שלהם.

פיתוח

Software Development Life Cycle = SDLC · מחזור פיתוח תוכנה

היכן אנחנו ב-SDLC

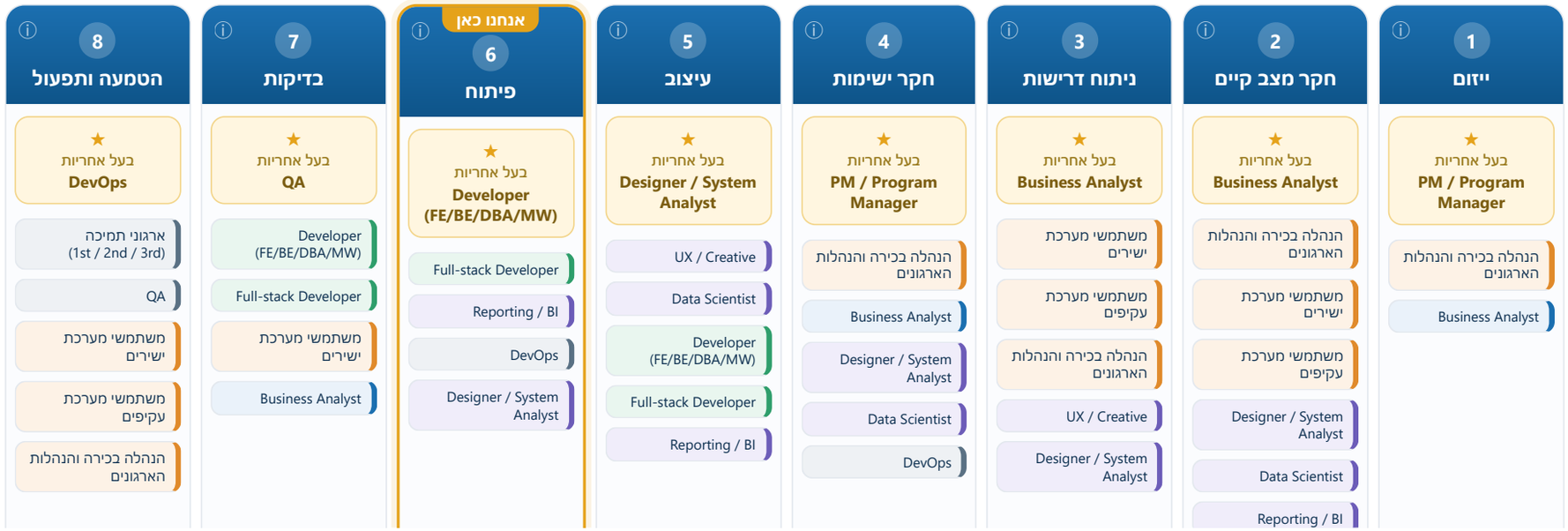
הצ׳יפ אומר פיתוח — אבל המתודולוגיה קובעת איך רצים בכל המסילה, לא רק בתחנה הזו

שלבי ה-SDLC — מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

לחצו על שלב או על תפקיד לקבלת הסבר קצר.



פיתוח

משבר התוכנה — The Software Crisis

הפער בין מה שאפשר להריץ לבין מה שאנחנו מצליחים לבנות באופן אמין

מה שקרה לחומרה

עקומה שעולה בלי הפסקה

- המחשב מהיר יותר — חוק מור
- המחשב זול יותר — מחירי רכיבים יורדים בקצב של 25%–40% בשנה
- צריכת אנרגיה קטנה פי אלפים לאותה עוצמה
- מזעור — הכול נכנס לכיס

בשנות ה-70 ההשקעה עברה מהחומרה לתוכנה — הרכיב שדורש בעיקר אנשים, ולכן הפך ליקר ביותר.

מה שקרה לתוכנה

הסימפטומים — ומה שמאחוריהם

- לא עומדים בדרישות — המערכת לא עושה מה שצריך
- אמינות — נופלת, מחשבת לא נכון
- עלות — חורגת פי כמה
- גמישות — כל שינוי קטן יקר ומסוכן

הסיבה: מורכבות. הרבה דרישות, הרבה מודולים, הרבה בדיקות, הרבה מפתחים ומשתמשים — ובעיות תקשורת ביניהם. מערכות נעשו מורכבות מכדי לנהל אותן.

ומה זה הוליד

התשובות שהתחום פיתח

- כלים וסביבות פיתוח
- שפות תכנות ופרדיגמות חדשות
- שפות מידול — UML, BPMN
- ארכיטקטורה
- שיטות בדיקה
- מתודולוגיות פיתוח

שימו לב מה לא פתר את המשבר: מחשבים מהירים יותר. מה שכן — שיטה. וכל הקורס הזה הוא ברשימה הזו.

והשאלה של 2026

אם המשבר נולד מכך שכתיבת תוכנה איטרת ויקרה — האם AI שמייצר קוד במהירות מסיים אותו? נחזור לזה בסוף ההרצאה. רמז: הוא לא מבטל את המורכבות — הוא מזיז את הצוואר.

פיתוח

BPMN = Business Process Model and Notation · מידול וסימון תהליכים עסקיים · UML = Unified Modeling Language · שפת מידול אחידה · AI = Artificial Intelligence · בינה מלאכותית

מה מתודולוגיה בעצם מחליטה

ארבע שאלות. כל מודל שנראה היום עונה עליהן — ולכן אפשר יהיה להשוות ביניהם

מתודולוגיה היא מסגרת שלפיה מפתחים את המערכת — הכלי שבו מנהל/ת הפרויקט מנווט/ת את תהליך הפיתוח. It is a framework in which the system is developed by.

1 מה גודל המנה?

כמה מהמערכת נבנה לפני שמישהו רואה משהו עובד? הכול בבת אחת — או פרוסה קטנה בכל פעם?

2 מה הסדר?

שלבים ברצף, כל אחד פעם אחת — או מעברים חוזרים שבכל אחד מהם עושים קצת מהכול?

3 מתי הלקוח רואה?

המפגש הראשון עם תוכנה עובדת: בסוף הפרויקט, או כל שבועיים? זה קובע כמה מוקדם נגלה שטעינו.

4 איך מטפלים בשינוי?

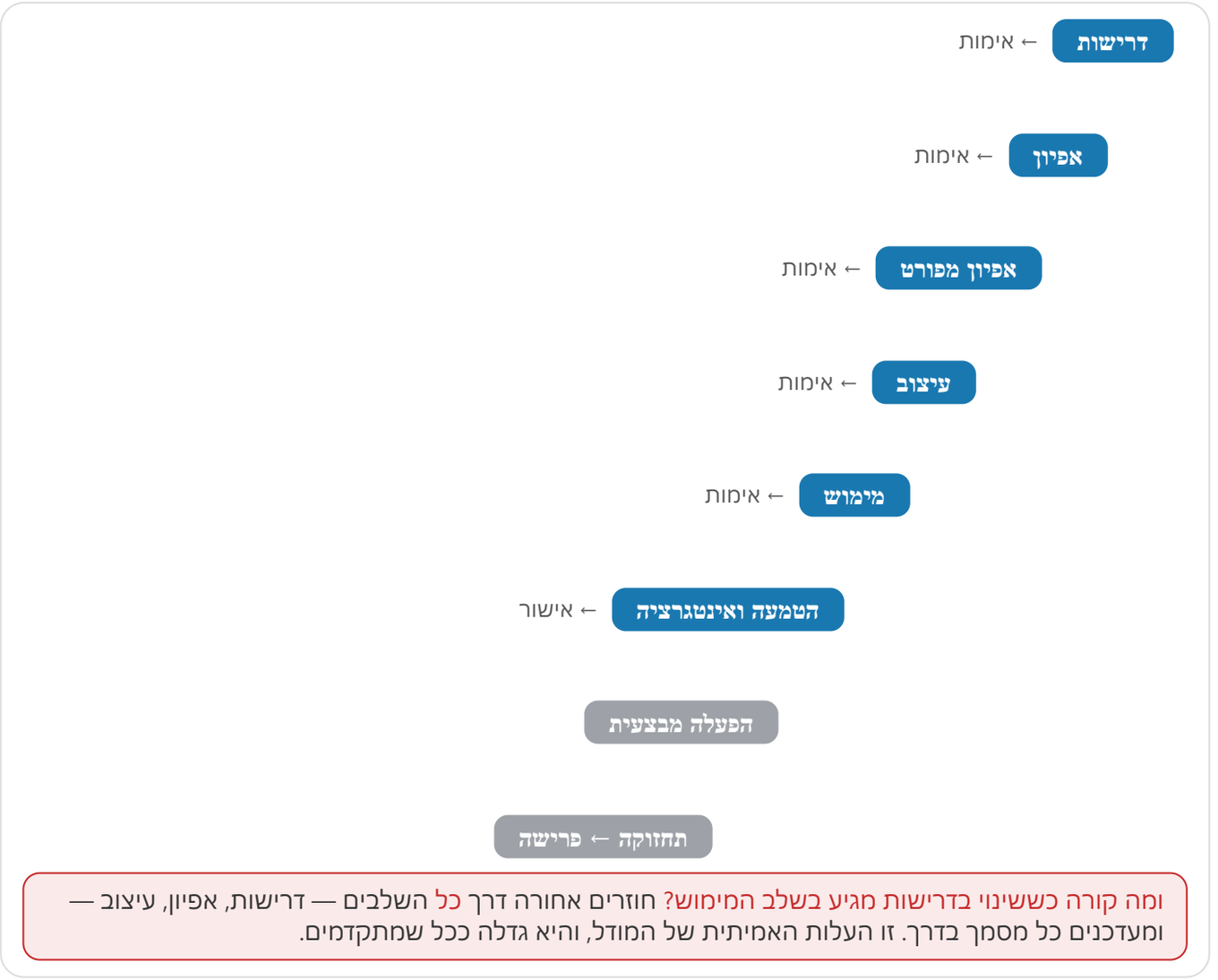
בקשת שינוי היא הפרעה שצריך לנהל ולתמחר — או שהיא התוכנית עצמה?

אין מודל שעונה "נכון" על כל הארבע. יש מודל שעונה נכון לפרויקט שלכם.

פיתוח

מודל מפל המים — Waterfall

המודל המובנה ביותר: שלבים סדורים, וכל שלב זורם אל הבא אחריו



הפלט של כל שלב הוא הקלט של הבא
מסמך הדרישות מזין את האפיון, האפיון מזין את העיצוב, העיצוב מזין את הקוד. שלב לא מתחיל לפני שקודמו אושר.

אימות בכל שלב
אחרי כל שלב בודקים שהתוצר נכון לפני שממשיכים. זו המשמעת של המודל — וגם מה שהופך אותו לאיטי.

יש חפיפה ופעילות גומלין
בפועל שלבים עוקבים חופפים בזמן, ושלב יכול לעדכן את קודמו. "מפל טהור" הוא ציור, לא פרויקט.

פיתוח

מפל מים — יתרונות, חסרונות, ומתי הוא דווקא נכון

המודל שהכי קל להתלוצץ עליו — ובכל זאת המודל הנכון לכמה סוגי פרויקטים

יתרונות

- תקורות מעבר נמוכות — פחות בזבוז בין שלב לשלב, פחות תיאום חוזר
- קל לניהול — ברור מי עושה מה ומתי, ומה מצב הפרויקט
- מסודר ומתועד — כל שלב מסתיים במסמך מאושר
- קל לתמחר ולחתום עליו בחוזה

בגלל השורה האחרונה — מכרזים ממשלתיים עדיין מנוסחים כך.

חסרונות

- הלקוח רואה את המערכת רק כשהיא מוכנה — זמן רב אחרי שהגדיר את הדרישות. ובינתיים גם הדרישות וגם הטכנולוגיה השתנו
- הגדרת יתר — כשצריך להחליט הכול מראש, מוסיפים "ליתר ביטחון" דרישות שאיש לא ישתמש בהן
- הלחץ מתרכז בסוף, אצל המתכנתים: כל עיכוב קודם נאכל מזמן הפיתוח

שינוי שמתגלה מאוחר עולה פי כמה משינוי שמתגלה מוקדם.

מתי כן להשתמש

- פרויקט קטן יחסית — כלל אצבע: עד 6 חודשים, עד 6 אנשים
- הדרישות מוגדרות היטב ולא משתנות
- קיימת כבר מערכת דומה — יודעים מה בונים
- קריטיות או רגולציה גבוהות — מערכות רפואיות, תעופה, בטיחות: שם התיעוד והאימות אינם תקורה אלא דרישה

מכשיר רפואי לא "נשחרר גרסה ונראה". יש דברים שאסור ללמוד תוך כדי.

1 • גודל המנה
כל המערכת בבת אחת

2 • הסדר
רצף, מעבר אחד

3 • הלקוח רואה
בסוף בלבד

4 • שינוי
הפרעה יקרה שמנוהלו

פיתוח

קטן, בינוני, גדול — קווים מנחים

אין הגדרה רשמית. אלה כללי אצבע — והם מספיקים כדי לדעת איזו שאלה לשאול

קריטריון	פרויקט קטן	פרויקט בינוני	פרויקט גדול
משך זמן	עד 6 חודשים	6–18 חודשים	מעל 18 חודשים
מספר עובדים	1–6 מפתחים	6–15 עובדים	מעל 15, כולל צוותים מקבילים
היקף דרישות	מוגדר היטב, מצומצם	בינוני, עשוי להשתנות	רחב ומורכב, דורש ניהול דרישות
מורכבות טכנולוגית	טכנולוגיות מוכרות	שילוב רכיבים או ממשקים	שילוב מערכות, טכנולוגיות חדשות
תלות חיצונית	כמעט ואין	מספר תלויות	תלות בגורמים וספקים רבים
חשיפה לסיכון	נמוכה	בינונית	גבוהה — רגולציה, שוק, ארגון
עלות	אלפים עד עשרות אלפים	מאות אלפים	מיליונים ומעלה

⚠ השורה שבאמת מכריעה

לא כמות האנשים ולא התקציב — אלא היקף הדרישות ויציבותן. פרויקט קטן עם דרישות שלא ברורות לאף אחד מסוכן יותר מפרויקט גדול עם דרישות יציבות.

ובקמפוס

כ-9 חודשים, צוות קטן, שני ממשקים למערכות האוניברסיטה, ודרישות שחלקן עוד לא ידועות (איך המטבח באמת יעבוד) — פרויקט בינוני. כלל ה-6/6 של מפל המים כבר לא מתקיים.

פיתוח

אב-טיפוס — Prototyping

מודל שנותן למשתמשים משהו לגעת בו מוקדם, כדי להבין מה הם באמת צריכים

אב-טיפוס אינו בעיקר מודל פיתוח — הוא כלי לאיסוף דרישות. הדרך הזולה ביותר לגלות שהדרישה שכתבנו איננה הדרישה שיש למשתמש.

מה זה נותן

- מודל בידי המשתמשים בשלב מוקדם של הפיתוח
- הבנה טובה בהרבה של הצורך האמיתי
- ללקוח יש דוגמה מוחשית של המוצר הסופי — במקום מסמך
- מתאים במיוחד כשהדרישות אינן מוגדרות היטב

"לא ידעתי מה אני רוצה עד שראיתי מה שלא רציתי." זה בדיוק מה שאב-טיפוס קונה.

יתרונות

- עוזר להבין את דרישות הלקוח — ומקטין את הסיכון הגדול ביותר בפרויקט
- הלקוח מקבל תוצר בתדירות גבוהה יחסית
- מגלה מוקדם התנגדויות והרגלי עבודה שלא צפינו

בקמפוס: תנו לצוות המטבח מסך דמה לשבוע — ותדעו הכול על ההיבט התרבותי.

חסרונות

- איכות מוצר ירודה — אב-טיפוס נבנה מהר, לא נכון
- עלות — במיוחד באב-טיפוס לזריקה, שנזרק לפי הגדרה
- הלקוח רואה מסך יפה וחושב שהמערכת כמעט מוכנה

החיסרון האחרון הוא ניהולי, לא טכני — וחפיל פרויקטים.

פיתוח

AI = Artificial Intelligence · בינה מלאכותית

שני סוגים של אב-טיפוס — ומה קורה לו בסוף

ההבדל ביניהם אינו טכני. הוא מה מתכננים לעשות איתו ביום שאחרי

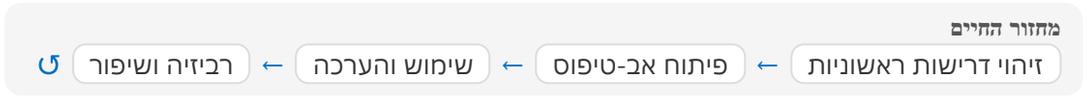
Throwaway prototyping
אב-טיפוס לזריקה

- מערכת זמנית שמסיימת את חייה בתום שלב הבחינה
- המטרה: להגביר את הקשר עם המשתמשים ולהבין מה הם רוצים מהמערכת
- שלבי הניתוח והעיצוב מתקצרים במכוון — חוסכים זמן על חשבון איכות, וזה בסדר
- הקמה מהירה, בחינה, ומסקנות

גורלו: נזרק — בכוונה. התוצר האמיתי הוא מסמך דרישות טוב יותר.

Evolutionary prototyping
אב-טיפוס מתפתח

- אב-טיפוס שעובר שינויים ובסוף התהליך הופך למערכת עצמה
- נבנה מתוך הנחה שישופר ויהפוך למערכת עובדת



גורלו: הופך למוצר. וכאן מתחילה הבעיה ←

⚠️ המלכודת — ומאיפה מגיע חוב טכני

אב-טיפוס נבנה מהר, לא נכון: בלי מבנה קוד תקין, בלי בדיקות, בלי טיפול בשגיאות. הכוונה תמיד לבנות מחדש "אחרי שנבין" — אבל אז מגיע לחץ של זמן וכסף, ואף אחד לא בונה מחדש. המערכת ממשיכה לרוץ שנים על יסודות שנועדו לשבוע. אם בוחרים evolutionary — צריך להחליט מראש מתי עוצרים ומשקיעים בבנייה נכונה.

פיתוח

פיתוח תוספתי — Incremental

מתאים למערכות גדולות ומורכבות. הרעיון: להקטין את הסיכון על ידי הקטנת הפרויקט

במקום פרויקט אחד גדול — סדרה של מיני-פרויקטים, שכל אחד מוסיף חלק מתפקד. אפשר להתחיל להשתמש בחלקים מהמערכת לפני שהפיתוח כולו הושלם.

שלוש דרכים לחתוך את מחזור החיים

סדרה של מפלי מים קטנים — כל תוספת עוברת את כל השלבים מקצה לקצה

מתכננים הכול, מבצעים בשלבים — פרויקט שלם עד סוף התכנון, ומשם מימוש מדורג

אוספים את כל הדרישות, ומשם בשלבים — דרישות פעם אחת, ואז תכנון וביצוע לכל תוספת

ההבדל ביניהן: כמה מקדימים מחליטים מראש. ככל שפחות — יותר גמישות, ויותר סיכון לאינטגרציה.

יתרונות

- בכל שלב מתקבל מוצר תפעולי — משהו שאפשר להשתמש בו, לא רק מסמך
- הטמעה מדורגת — הארגון מתרגל לשינוי בקצב שהוא סופג
- ערך עסקי מגיע מוקדם, במקום להמתין לסוף
- סיכון קטן יותר: טעות פוגעת בתוספת אחת, לא בפרויקט

חסרונות

- רמת סיבוכיות עולה — כל תוספת צריכה להשתלב במה שכבר קיים ולא לשבור אותו
- נדרשת ארכיטקטורה שתחזיק את כל התוספות שיבואו — ואותה כן צריך לחשוב מראש
- יותר תקורות מעבר: אינטגרציה, בדיקות ומסירה בכל סבב

הכשל הקלאסי: המודל מתדרדר ל"בנה ותקן" — מוסיפים חתיכות בלי תכנון, עד שאי אפשר להוסיף עוד.

1 • גודל המנה
תוספת אחת בכל פעם

2 • הסדר
מעברים חוזרים על השלבים

3 • הלקוח רואה
בסוף כל תוספת

4 • שינוי
נכנס לתוספת הבאה

פיתוח

תוספתי מול איטרטיבי — ההבדל שאי אפשר לבטל

שניהם "בונים בשלבים". השאלה היא אם בסוף כל שלב יש משהו שאפשר להשתמש בו

תוספתי
Incremental
מוסיפים בכל פעם חלק חדש של המערכת. כל חלק מפותח מההתחלה עד הסוף.

- **גלגל** — ✗ לא שימושי
- **גלגלים ושלדה** — ✗ לא שימושי
- **+ מרכב ומנוע** — ✗ לא שימושי
- **מכונית** — ✓ סוף סוף

איטרטיבי
Iterative
כל הפונקציות קיימות מההתחלה, ומשתפרות בכל סבב — גרסה בסיסית, ואז שדרוגים.

- **סקייטבורד** — ✓ מגיע ליעד
- **קורקינט** — ✓ מהר יותר
- **אופניים** — ✓ נוח יותר
- **מכונית** — ✓ מצוין

ובפועל — שניהם יחד
פרויקטים אמיתיים עושים את שניהם: מוסיפים יכולות חדשות (תוספתי) ומשפרים קיימות (איטרטיבי) באותו סבב. בקמפוס: סבב שמוסיף "תשלום" הוא תוספתי; סבב שמחליף אישור ידני של הצוות בהתראה אוטומטית — איטרטיבי. השאלה שנשארת בכל סבב: מה הסטודנט יכול לעשות עכשיו שלא יכול היה אתמול?

פיתוח

מודל הספירלה — Spiral

המודל היחיד שבו סדר העבודה נקבע לפי סיכון, ולא לפי מבנה המערכת

קודם פותרים את מה שהכי מסוכן. אפיון מפורט של רכיבים ברמת סיכון נמוכה — נדחה לשלב מאוחר יותר. אם מזהים סיכון חדש, אפשר לחזור אחורה.

1 תכנון הסבב

קביעת מטרות לחלק המוצר שבמוקד (ביצועים, פונקציונליות, קבלת שינוי), חלופות להשגתן (עיצוב A מול B, שימוש חוזר מול רכישת רכיב), ואילוצים (עלות, לו״ז, ממשק).

2 ניתוח סיכונים

השוואת החלופות לאור האילוצים, וזיהוי תחומי אי-ודאות. פיתוח אסטרטגיה להתמודד: אב-טיפוס, סימולציה, benchmarking, שאלונים, מודל אנליטי.

3 פיתוח ובדיקה

פיתוח מענה למטרות שהוגדרו — ובדיקתו. כאן, ורק כאן, נכתב קוד — ורק לחלק שהסבב הזה עוסק בו.

4 הערכה ותכנון הסבב הבא

בדיקת הסיכונים שנותרו, והחלטה מה מפתחים בסבב הבא — ובאיזו שיטה: אב-טיפוס מתפתח, מפל מים, או משהו אחר.

אורך סבב

בין מספר חודשים לשנתיים. זה לא ספרינט — זה מחזור ניהולי כבד.

מכיל מודלים אחרים

אפשר לשלב בתוך סבב אלמנטים של מפל מים ושל אב-טיפוס. הספירלה היא מסגרת-על, לא שיטת פיתוח.

למה זה עובד

כי מה שהורג פרויקטים גדולים אינו מה שקשה — אלא מה שלא ידענו. הספירלה מכריחה לתקוף את אי-הידיעה ראשונה.

פיתוח

הספירלה — מה היא לא, ומתי היא כן

התרשים המפורסם קיבל הרבה פרשנויות שגויות. שלוש מהן חוזרות תמיד

✗ "הספירלה מגדירה תהליך אחד מסודר"

זו סדרה של סבבים, וכל אחד יכול להיראות אחרת לגמרי — תלוי במטרות ובסיכונים של אותו סבב.

✗ "זה פשוט אוסף של מפלי מים קטנים"

הדגש אינו על הפיתוח אלא על ניהול הסיכונים ועל מעבר גמיש בין שלבים. מפל קטן אינו שואל "מה הכי מסוכן".

✗ "כל רביע תמיד מבוצע"

הפעולות הן פוטנציאליות. סבב שבו אין אי-ודאות מהותית לא צריך אב-טיפוס ולא סימולציה.

יתרונות

- מתאימה לפרויקטים גדולים מאוד ולחוסר ודאות גבוה
- תוכנה ממומשת כבר בשלב מוקדם
- ניתוח סיכונים נרחב ומובנה
- מזעור נזקים — עשויה להוביל להחלטה לעצור את הפרויקט מוקדם, וזה יתרון

חסרונות

- מחייבת צוות מיומן בניתוח סיכונים — לא כל ארגון מסוגל
- עבודה ש"נזרקת לפח" — אב-טיפוסים וסימולציות שאינם תורמים ישירות לתוצר
- משך הפרויקט הכולל קשה להערכה, וזה בעייתי מול תקציב שנתי
- לא מתאימה לפרויקטים קטנים בסיכון נמוך — התקורה בולעת אותם

מתי כן

- פרויקט גדול ויקר, שכישלונו יקר מאוד
- טכנולוגיה או דרישות שאף אחד עוד לא ניסה
- יש סיכון שיכול להפיל את הפרויקט כולו, ועדיף לגלות אותו מוקדם
- לארגון יש היכולת והסמכות לעצור באמצע

1 • גודל המנה
סבב לפי סיכון, לא לפי יכולת

2 • הסדר
חוזר — הכי מסוכן ראשון

3 • הלקוח רואה
אב-טיפוסים מוקדם

4 • שינוי
סיכון חדש ← סבב חדש

פיתוח

AI = Artificial Intelligence · בינה מלאכותית

Agile — פיתוח זריז וגמיש

לא "עוד מודל" — אלא טענה על טבע התוכנה, וסדרה שלמה של שיטות שנגזרות ממנה

הטענה שממנה הכול נובע

לא ניתן להגדיר במלואה תוכנה מסוימת קודם לפיתוחה בפועל.

ואם הטענה נכונה — מה נובע ממנה?

- אין טעם להשקיע חודשים במפרט מלא — הוא יתגלה כשגוי חלקית בכל מקרה
- עדיף לספק תוצרים במהירות ולגלות מה נכון מהמציאות
- דרישות ופתרונות מתפתחים תוך כדי עבודה משותפת, בקבוצות אוטונומיות

שימו לב מה זה זה **לא** אומר: לא "בלי לתכנן". אומר: לתכנן ברזולוציה שמתאימה לוודאות שיש.

מה משתנה בפועל

- מנה קטנה — שבועיים עד חודש, ובסופה תוכנה עובדת
- הלקוח בפנים — לא לקוח שמאשר מסמך, אלא שותף יומיומי
- הצוות מחליט איך לבצע — לא מקבל תוכנית מלמעלה
- שינוי מתקבל בברכה, גם בשלב מתקדם

וזו משפחה, לא שיטה

Agile הוא **אוסף ערכים ועקרונות**. מתחתיו יושבות שיטות ממשיות שאותן באמת מיישמים:

Scrum · **Extreme Programming (XP)** · **Kanban** · **AUP** · **Crystal Clear** · **ASD** · **DSDM** · **RAD**

"אנחנו עובדים **אג'ייל**" אינו משפט מלא. השאלה היא **איזו שיטה**, ואיך היא מיושמת.

1 • גודל המנה
שבועיים עד חודש

2 • הסדר
חוזר, קצר וקבוע

3 • הלקוח רואה
בסוף כל סבב

4 • שינוי
מתקבל בברכה — הוא פיתוח

המנשר האג׳ילי — ארבעה ערכים

ארבעה זוגות. שני הצדדים בעלי ערך — אבל כשצריך להכריע, שמאל גובר

אנשים ואינטראקציות Individuals and interactions שיחה של עשר דקות עדיפה על שלושה מיילים ועל כלי ניהול משימות	**גובר על**	**תהליכים וכלים** processes and tools אבל בלי תהליך אין קצב, ובלי כלים אין שקיפות
תוכנה עובדת Working software ״80% מהקוד נכתב״ אינו התקדמות. פיצ׳ר שרץ אצל משתמש — כן	**גובר על**	**תיעוד מקיף** comprehensive documentation אבל תרשים שחוסך שבועיים אי-הבנה הוא תקשורת, לא ניירת
שיתוף פעולה עם הלקוח Customer collaboration הלקוח בפנים כל שבועיים, לא מאשר מסמך פעם ברבעון	**גובר על**	**משא ומתן חוזי** contract negotiation אבל בלי חוזה אין מבחני קבלה — ראו הרצאה 10
תגובה לשינוי Responding to change תוכנית שלא השתנתה בשלושה חודשים — סימן שלא למדנו כלום	**גובר על**	**עמידה בתוכנית** following a plan אבל בלי תוכנית אי אפשר לתקצב ואי אפשר לדווח

והמשפט שכולם שוכחים — והוא חלק מהמנשר

״גם לפריטים שמימין יש ערך — אנחנו פשוט מעריכים את אלה שמשמאל יותר.״

while there is value in the items on the right, we value the items on the left more

פיתוח

העקרונות שבאמת משנים התנהגות

מתוך 13 עקרונות המנשר — אלה שאפשר לראות אם צוות מקיים אותם או לא

אספקה

- העדיפות העליונה — לספק מוקדם וברציפות תוכנה בעלת ערך ללקוח
- אספקה תדירה של תוכנה עובדת: משבועות ועד חודשים, בשאיפה לקצר
- תוכנה עובדת היא יחידת המידה העיקרית להתקדמות

ולכן בפועל: "סיימנו 80% מהקוד" אינו דיווח התקדמות. "הפיצ'ר עובד אצל משתמש" — כן.

שינוי

- מקבלים בברכה שינויים בדרישות — אפילו בשלב מתקדם
- תהליכים ברי-שינוי הם יתרון תחרותי ללקוח, לא נזק
- אנשי פיתוח והמשתמש העסקי עובדים יחד יום-יום לאורך הפרויקט

ולכן בפועל: אין "ועדת שינויים" שמתכנסת פעם בחודש. יש בקלוג שמתועדף מחדש כל סבב.

אנשים

- בונים פרויקטים סביב יחידים בעלי מוטיבציה: תנו סביבה, ענו על צרכים, וסמכו עליהם
- שיחה פנים אל פנים היא הדרך היעילה ביותר להעביר מידע
- ארכיטקטורה, עיצוב ודרישות מגיעים מצוותים עצמאיים
- קצב בר-קיימא — לא לרוץ ולקרוס

ולכן בפועל: "סמוך עליהם" הוא הקשה מכולם. ארגונים מאמצים טקסים ולא מוותרים על שליטה.

מקצועיות

- תשומת לב מתמדת למצוינות טכנית ולעיצוב טוב — הם מה שמאפשר להישאר זריזים
- פשטות חיונית — האומנות למקסם את כמות העבודה שלא תיעשה
- הצוות עוצר, חושב איך להיות אפקטיבי יותר, ומשנה את התנהגותו

ולכן בפועל: עיצוב טוב אינו מותרות באג'ייל — הוא התנאי לכך שהסבב הבא י

פיתוח

אחוזי הצלחה — ואיך לקרוא אותם

המספר שכולם מצטטים. שווה להכיר אותו — ולדעת מה הוא לא אומר

גודל הפרויקט	Agile — הצליח	Waterfall — הצליח
גדול	18%	3%
בינוני	27%	7%
קטן	58%	44%
כלל הפרויקטים	39%	11%

Standish Group, CHAOS Report 2015. "הצלחה" = עמידה בזמן, בתקציב ובתכולה.

מה כן אפשר ללמוד מזה

שתי מגמות יציבות שחוזרות בכל דוח: ככל שהפרויקט גדול יותר — שיעור ההצלחה נמוך יותר, בשתי השיטות. והפער לטובת אג׳ייל גדל ככל שהפרויקט גדול. שתי המגמות מצביעות לאותו כיוון: מנות קטנות.

⚠ **מה נחשב "הצלחה"**

בזמן, בתקציב ובתכולה. לא "הביא ערך לארגון". מערכת שסופקה בדיוק כמתוכנן ואיש לא משתמש בה — נספרת כהצלחה.

⚠ **הטיית בחירה**

מי בוחר אג׳ייל? לרוב ארגונים וצוותים שכבר בשלים יותר. ייתכן שהשיטה מסמנת צוות טוב יותר, ולא גורמת לו.

⚠ **הגדרות שנעות**

בפרויקט אג׳ילי התכולה משתנה במכוון. "עמידה בתכולה" פשוט אינה אותה מדידה בשתי השיטות.

⚠ **מי מדד**

הנתונים של גוף פרטי, המתודולוגיה אינה פתוחה לביקורת מלאה, והדוחות שנויים במחלוקת בספרות. לצטט בזהירות.

פיתוח

Scrum — המסגרת

השיטה האג׳ילית הנפוצה ביותר. שלושה סוגי דברים, וזה הכול

ההנחה שביסודה: פיתוח תוכנה הוא בעיה אמפירית — לומדים אותה מתצפית וניסיון — ולא בעיה שאפשר לפתור בחיזוי ובתכנון מראש. מכאן הכול נגזר.

מה זה אומר בפועל

- לספק את הערך העסקי הגבוה ביותר בזמן הקצר ביותר
- לבחון תוכנה עובדת שוב ושוב, כל שבועיים עד חודש
- העסק קובע עדיפויות; הצוות מחליט איך לממש
- צוותים קטנים, רב-תחומיים ובהכוונה עצמית — לא פיקוח מלמעלה
- בסוף כל סבב אפשר לשחרר כמו שזה או להמשיך לשפר

פותחה ע״י קן שוואבר וג׳ף סאתרלנד, על בסיס טקאוצ׳י ונונקה (1986): צוותים קטנים ורב-תחומיים משיגים תוצאות טובות יותר בפיתוח מוצרים חדשים.

בעלי תפקידים
Roles

- **Product Owner**
- **Scrum Master**
- **Team**

מי מחליט, מי מגן ומי בונה

טקסים
Ceremonies

- **Sprint planning**
- **Daily scrum**
- **Sprint review**
- **Sprint retrospective**

מתי נפגשים, ולשם מה

תוצרים
Artifacts

- **Product backlog**
- **Sprint backlog**
- **Burndown chart**

מה כתוב, ומי — פיתוח

מחזור הספרינט

מרשימת הדרישות ועד תוספת עובדת — ושוב, כל שבועיים עד חודש

בקלוג המוצר
Product Backlog
כל מה שרוצים במערכת, מתועדף — באחריות ה-Product Owner

תכנון ספרינט
Sprint Planning
בוחרים מטרה, ומושכים ממנה פריטים לבקלוג הספרינט

ספרינט
4–2 שבועות · אורך קבוע

פגישה יומית · Daily Scrum
15 דקות, בעמידה. מה עשיתי אתמול · מה אעשה היום · מה חוסם אותי.
לאורך כל הספרינט המוצר מעוצב, מקודד ונבדק.

בסופו: תוספת עובדת — Increment

סקירת ספרינט
Sprint Review
מציגים את מה שנעשה, ומעדכנים את בקלוג המוצר לפי מה שנלמד

רטרוספקטיבה
Sprint Retrospective
איך עבדנו — לא מה בנינו. ומה נשנה בספרינט הבא

כלל 1 — אורך קבוע
אותו מספר שבועות בכל ספרינט. אורך קבוע מכניס את הצוות לקצב, ומאפשר לאמוד כמה נכנס לספרינט על סמך ניסיון ולא על סמך ניחוש.

כלל 2 — אין שינוי בתוך ספרינט
ברגע שהספרינט התחיל, לא נכנסות אליו דרישות חדשות. בקשה שמגיעה באמצע — הולכת לבקלוג ותתועדף לספרינט הבא. זה מה שהופך שבועיים להבטחה ולא לכוונה.

פיתוח

Scrum — בעלי התפקידים

שלושה תפקידים בלבד. ומי שמנהל את הפרויקט — לא נמצא ברשימה

Product Owner
בעל/ת המוצר — מייצג/ת את הלקוח

- חלק מצוות הפיתוח, ואחראי/ת על הקשר עם הלקוח וייצוג צרכיו
- מגדיר/ה את צרכי המוצר
- מחליט/ה על תאריכי שחרור ותוכן
- אחראי/ת ליצירת הערך העסקי של המוצר (ROI)
- מתעדף/ת את בקלוג המוצר לכל ספרינט
- מקבל/ת או דוחה את התוצר בסוף הספרינט

מחליט/ה מה ייבנה ובאיזה סדר — לעולם לא איך.

Scrum Master
שומר/ת השיטה

- מייצג/ת את ההנהלה בפרויקט ומתאם/ת בין הצוות לארגון
- אחראי/ת לשמירה על עקרונות ושיטת העבודה של Scrum
- הסרת מכשולים — טכנולוגיים, ארגוניים, תקשורתיים
- וידוא שעבודת הצוות יעילה
- יצירת שיתוף פעולה בין כל חלקי הפרויקט
- הגנה על הצוות מפני הפרעות חיצוניות

כמו מאמן/ת הקבוצה: לא משחק/ת בעצמו/ה — דואג/ת שהשחקנים יצליחו.

Team
צוות הפיתוח

- 5–9 אנשים, בדרך כלל
- חוצה תחומים: מתכנתים, בודקים, UX, IT, עיצוב
- משרה מלאה — לא עובד על משימות חיצוניות במקביל (למעט מקרים שאין ברירה, כמו DBA משותף)
- מתארגן לבד — אין תארים
- תחלופה רק בין ספרינטים, ועדיף בכלל לא

מחליט איך ייבנה, ומתחייב על כמה [illegible].

פיתוח

Scrum — הטקסים

ארבע פגישות. לכל אחת מטרה אחת — וכל אחת מהן מתדרדרת בקלות למשהו אחר

Sprint Planning תכנון ספרינט

בתחילת כל ספרינט · הצוות + PO + SM

- קלטים: קיבולת הצוות, בקלוג המוצר, תנאים עסקיים, מצב המוצר ומגבלות טכנולוגיות
- חלק א׳ — תעדוף: בוחרים מטרת ספרינט אחת ברורה
- חלק ב׳ — תכנון: איך משיגים אותה, בונים את בקלוג הספרינט ומעריכים בשעות

לא רשימת משאלות של ההנהלה. הצוות הוא זה שמתחייב — לפי קיבולת אמיתית.

Daily Scrum פגישה יומית

כל יום · 15 דקות · בעמידה · כולם מוזמנים, רק הצוות מדבר

- מה עשיתי אתמול
- מה אעשה היום
- מה חוסם אותי

לא דיווח סטטוס לסקראם מאסטר, ולא מקום לפתור בעיות. אלה התחייבויות בפני עמיתים; מי שחסום נשאר אחרי הפגישה.

Sprint Review סקירת ספרינט

בסוף הספרינט · 2–4 שעות · + בעלי עניין

- הצוות מציג את התוספת שפותחה
- בודקים מה מוכן לפי הגדרת Done, ומה לא
- מעדכנים את בקלוג המוצר לפי מה שנלמד, וקובעים תאריך לתכנון הבא

לא מצגת. מראים תוכנה שרצה. ואם הבקלוג לא השתנה בסופה — כנראה לא באמת למדנו כלום.

Sprint Retrospective רטרוספקטיבה

אחרי הסקירה · הצוות בלבד

- מה עבד טוב?
- מה אפשר לשפר?
- איך נשפר את זה בספרינט הבא — פעולה אחת קונקרטית

לא על מה בנינו אלא על איך עבדנו, ולא מקום לשפוט אנשים. זו הפגישה הראשונה שמבטלים כשלוחצים — וזו הטעות.

פיתוח

Scrum — התוצרים, והבקלוג של הקמפוס

שלוש רשימות. ואת הראשונה שבהן — כבר כתבנו, בהרצאות 4–5

Product Backlog
בקלוג המוצר

רשימת כל העבודה הרצויה בפרויקט — הדרישות. כל פריט בעל ערך למשתמש או ללקוח. מתועדף ע״י ה-PO, ומתועדף מחדש בתחילת כל ספרינט. רשימה חיה, לא מסמך מאושר.

Sprint Backlog
בקלוג הספרינט

מה שהצוות התחייב לספרינט הזה, מפורק למשימות. חברי הצוות בוחרים בעצמם — עבודה לעולם לא מוקצית. הזמן שנותר מתעדכן כל יום, וכל חבר צוות יכול להוסיף, לשנות או למחוק.

Burndown Chart
גרף שריפה

כמה עבודה נותרה בכל יום של הספרינט. ציר אנכי — עבודה שנותרה; אופקי — ימים. קו מתוכנן מול קו בפועל. כלי של הצוות לעצמו, לא כלי בקרה של ההנהלה.

בקלוג המוצר — מערכת ההזמנות בקמפוס

פריט — סיפור משתמש	הערכה	מקור
כסטודנט/ית, אני רוצה לראות את התפריט של הקפיטריה, כדי לדעת מה יש היום	3	עיון בתפריט
כסטודנט/ית, אני רוצה להזמין מנות ולקבוע שעת איסוף, כדי לא לעמוד בתור	8	ביצוע הזמנה
כעובד/ת מטבח, אני רוצה לראות את תור ההזמנות לפי עמדה, כדי לדעת מה להכין	5	תור הכנה
כעובד/ת מטבח, אני רוצה לסמן מנה כמוכנה, כדי שהסטודנט/ית יידע/תדע לבוא	3	סימון מוכן
כסטודנט/ית, אני רוצה לשלם בכרטיס האוניברסיטה, כדי לא להזין כרטיס אשראי	13	תשלום
כעובד/ת דלפק, אני רוצה לאמת שההזמנה שולמה ושייכת לסטודנט/ית	8	אימות איסוף
כסטודנט/ית, אני רוצה לסנן מנות לפי העדפה תזונתית	5	עיון בתפריט
כמנהל/ת קפיטריה, אני רוצה לעדכן שמנה אזלה, כדי שלא יזמינו אותה	3	ניהול מלאי
כמנהל/ת קפיטריה, אני רוצה דוח מכירות יומי, כדי לתכנן הזמנות מספקים	8	דוח יומי
... ועוד פריטים שטרם פורקו	21	—

שימו לב לעמודה השמאלית: כל פריט הוא תרחיש שימוש מהרצאות 4–5, כתוב כסיפור משתמש ומתומחר. הבקלוג אינו תוצר חדש — הוא מודל תרחישי השימוש, מסודר לפי עדיפות.

פיתוח

Extreme Programming — ולמה הוא לא מתחרה ב-Scrum

שיטה אג׳ילית שמדברת דווקא על מה ש-Scrum שותק לגביו: איך כותבים את הקוד

XP — קנט בק וורד קנינגהאם, 2000

מבוססת על תקשורת, פשטות ומשוב מהלקוח. מתאימה במיוחד לסביבות עם דרישות משתנות וקצב עבודה גבוה.

הקשבה | עיצוב | קידוד | בדיקות

- הלקוחות כותבים סיפורי משתמש — בשפת הלקוח, לא טכנית; לכל סיפור הערכת מאמץ (למשל 1–3 ימי עבודה)
- הסיפורים משמשים גם בסיס לבדיקות קבלה
- מסירת גרסה שמישה כל 1–3 שבועות; לפני כל סבב — פגישת תכנון
- עבודה בצוותים קטנים, ולעיתים בזוגות (pair programming)
- משימות קטנות, מעקב יומי (stand-up), בדיקות אוטומטיות ושיפור מתמיד

Scrum מול XP

נושא	Scrum	XP
מיקוד	ניהול הצוות והתהליך	איכות הקוד והתהליך הטכני
הנחיות לכתיבת קוד	אין — רק ניהול משימות	יש — בדיקות, תכנות בזוגות, עיצוב פשוט
שיתוף הלקוח	בתכנון ובסקירה	שותף פעיל לאורך כל הדרך
כלי ניהול	Backlog, Sprint, Burndown	Stories, Iterations, Pair Programming

תסתכלו על השורה השנייה: Scrum לא אומר דבר על איך לכתוב קוד, ו-XP כן. הן לא חלופות — הן משלימות. רוב הצוותים בשטח מריצים Scrum כמסגרת, עם פרקטיקות XP בפנים: בדיקות אוטומטיות, אינטגרציה רציפה וסקירת קוד.

פיתוח

כשזה גדל — Scaled Agile

סקראם תוכנן לצוות אחד של 5–9 אנשים. מה עושים כשיש שנים-עשר צוותים?

מעל כ-20 מפתחים נוטים לאמץ גישות היברידיות. הבעיה שהן פותרות: תיאום בין צוותים — תלויות הדדיות, ממשקים משותפים, ומסירה בקצב אחיד.

רמת הצוות
Team Level

צוותי Scrum רגילים — PO, Scrum Master, מפתחים ובודקים. עובדים בספרינטים, ובסופם תכנון, דמו ורטרוספקטיבה. בדיוק מה שלמדנו.

מתכננים בסיפורי משתמש
אופק: ספרינט

רמת התכנית
Program Level · ART

קבוצה של 5–12 צוותים שעובדים יחד לספק פתרון בקצב אחיד (Agile Release Train). תיאום, הדגמת מערכת משותפת, ותפקידים רוחביים — UX, ארכיטקטים.

מתכננים בפיצ׳רים
אופק: PI — כמה ספרינטים

רמת הפורטפוליו
Portfolio Level

כאן מחליטים על כיווני המוצר וההשקעה: תקציבים, מדדים ואסטרטגיה ארגונית. ניהול באמצעות Kanban להשקעות עיקריות.

מתכננים בEpics
אופק: שנה ומעלה

המסגרות המוכרות: SAFe · LeSS · Nexus · Disciplined Agile Delivery · Scrum at Scale · Enterprise Scrum

⚠️ **וגם ביקורת: ככל שמוסיפים שכבות תיאום, חוזרת ההיררכיה שאגיל ניסה להסיר.**

פיתוח

ואיפה נמצאים התרשימים שלנו?

השאלה שאתם מחזיקים מהרצאה 2 — ועכשיו יש לכם את המילים לשאול אותה

השאלה

אם היום עובדים אג׳ייל — BPMN, תרחישי שימוש, תרשים מחלקות ותרשים מצבים הם שריד של מפל מים?

התוצר	בפרויקט מפל מים	בפרויקט Scrum
תרשים BPMN הרצאה 2	פעם אחת, בתחילת הפרויקט, לכל התהליכים בארגון	פעם אחת לתהליך הליבה; תהליך משני ממופה כשמגיעים אליו
קטלוג דרישות הרצאה 3	מסמך שלם ומאושר לפני העיצוב. שינוי = ועדת שינויים	בקלוג המוצר — רשימה חיה, מתועדפת מחדש בכל ספרינט
תרחישי שימוש הרצאות 4–5	כולם, מפורטים מראש, לפני שנכתבה שורת קוד	תרחיש = פריט בקלוג. המפרט המפורט נכתב בספרינט שבו הוא מפותח
תרשים מחלקות הרצאות 6–7	מודל התחום המלא, מאושר, לפני הקידוד	שלד ארכיטקטוני מראש + הפרוסה שהספרינט צריך. המודל צומח
תרשים מצבים הרצאה 8	לכל אובייקט בעל מחזור חיים, מראש	כשהאובייקט מקבל מחזור חיים אמיתי — בספרינט שמוסיף לו מצבים
חקר ישימות הרצאה 10	פעם אחת, בשלב 4, לכל המערכת	פעם אחת בגדול — ועוד החלטות ״לבנות או לקנות״ לכל רכיב, תוך כדי

התשובה

אף תוצר לא נעלם. מה שמשתנה הוא גודל המנה והתזמון: לא ״הכול מראש״ אלא ״מה שצריך, כשצריך״. אג׳ייל אינו ׳בלי עיצוב׳ — הוא עיצוב במנות קטנות, בזמן אמת. ומודל שנבנה בפרוסות דורש דווקא יותר משמעת מידול, לא פחות — כי כל פרוסה חייבת להשתלב במה שכבר קיים.

פיתוח

BPMN = Business Process Model and Notation · מידול וסימון תהליכים עסקיים

תרגול: איזו מתודולוגיה — ולמה

חמישה פרויקטים. קודם מחליטים בכיתה, ורק אחר כך הופכים את הכרטיס

בכל כרטיס יש תכונה אחת שמכריעה. חפשו אותה — היא כמעט תמיד קשורה ליציבות הדרישות או למחיר של טעות.

אתר מיפ (V-Model)

מתאים כש: הדרישות מוגדרות היטב מראש

דרישות ברורות וקבועות כבר מראש, ואי אפשר "לשחרר ולראות". תיקון באיפיון אחרי הקוד — הם יקרים.

Scrum • איטרציה-והדפה

מתאים כש: דרישות לא יציבות

הקלק מוצלחת נמדדת רק כשמשתמשים אמיתיים יגלו בסוף. מוכנה קצרות ושינוי מהיר. משקיעים רוצים מוצרים.

והדפה + תכנית

מתאים כש: משימה אינדיבידואלית

מוגדר אחר מוגדר, כל שחריגה יסמן את הסיום. הדרישות דומה ידועות (הפוצר כלים) — הקושי הוא אלגו והתמידים.

מפרקת

מתאים כש: אי-ודאות מהדרגתית

הסיכון העיקר איזו "מה נבנה" אלא "האם זה בכלל אפשרי". הוקמים את הכי-קטן ראשון, כמותיבציות ובאב-סיפוסים.

אב-טיפוס • ניתוח שיטי ומשותף

מתאים כש: צוות נסמך יחידני

אין צוות, אין כלום מעבר, ואין סיכון. כל מהנדסים כאן היא הקוד שעדיין מהפרויקט לצמוד.

שימו לב שאף כרטיס לא הוכרע לפי "מה מודרני" — אלא לפי יציבות הדרישות ומחיר הטעות.

פיתוח

AI = Artificial Intelligence · בינה מלאכותית

ואיך נריץ את פרויקט הקמפוס

ההמלצה מהרצאה 10 אמרה "להעלות בשלבים". הנה מה זה אומר בפועל

ההחלטה

איטרטיבי-תוספתי בסגנון Scrum • ספרינטים של שבועיים • תוספת ראשונה = קפיטריה אחת

גודל מנה — שבועיים · סדר — חוזר · הלקוח רואה — בסוף כל ספרינט · שינוי — נכנס לבקלוג ומתועדף

ספרינט 1

סטודנט/ית מזמין/ה — ולא עומד/ת בתור

עיון בתפריט (3) · הזמנה וקביעת שעת איסוף (8) · תור הזמנות לפי עמדה (5)
ללא תשלום — משלמים בדלפק כרגיל. ללא התראות — הצוות מסמן מוכן ידנית. **זה הסקייטבורד.**

ספרינט 2

ההזמנה נסגרת בלי לגעת בכסף בדלפק

תשלום בכרטיס האוניברסיטה (13 — מפורק) · אימות איסוף (8)
כאן יושב הממשק למערכת הזיהוי — הסיכון הגבוה ביותר, ולכן מוקדם.

ספרינט 3

המטבח וההנהלה מקבלים את שלהם

סימון מוכן והתראה לסטודנט (3) · עדכון "אזל" (3) · סינון תזונתי (5) · דוח מכירות יומי (8)
ואז: פיילוט בקפיטריה אחת, ומשם הרחבה.

✗ למה לא מפל מים

כלל 6/6 לא מתקיים (כ-9 חודשים), וחלק מהדרישות באמת לא ידועות — איך המטבח יעבוד בשעת עומס יתברר רק בשטח. מסירה אחת בסוף = לגלות בחודש התשיעי.

✗ למה לא אב-טיפוס בלבד

המערכת נוגעת בכסף ובזהות של 5,000 סטודנטים. אב-טיפוס מתפתח שהפך למוצר הוא בדיוק המלכודת שדיברנו עליה.

✓ ומה זה נותן לנו

אחרי שישה שבועות יש משהו שסטודנטים משתמשים בו.
הסיכון הגדול (הממשק) נבדק בספרינט 2 ולא בחודש התשיעי.
ואם התקציב ייחתך — יש מערכת חלקית שעובדת, לא חצי פרויקט.

פיתוח

SDLC = Software Development Life Cycle · מחזור פיתוח תוכנה

🤖 מה ה-AI עושה למחזור — ⚠️ ומה נעשה קשה יותר

בתחילת ההרצאה שאלנו אם AI מסיים את משבר התוכנה. הנה התשובה

ה-AI הוא המתכנת שמקבל מפרט. ולכן הוא מוזיל בצורה דרמטית את מה שהוא עושה — ומשאיר את כל השאר במקום, או מחמיר אותו.

✓ מה נעשה זול ומהיר

עלות הסבב צונחת
"תוספת עובדת" שדרשה שבועיים יכולה לדרוש ימים. וכשסבב זול — משתלם לעשות סבבים קצרים יותר. המטוטלת נעה עוד לכיוון האיטרטיבי.

אב-טיפוס לזריקה כמעט חינם
החיסרון הגדול שלו היה שהוא עולה ונזרק. כשהוא נבנה בשעה — אפשר לבנות שלושה ולבחור. וגם פוחת הפיתוי להפוך אב-טיפוס למוצר.

"בואו ננסה" הופך לזול
ניתוח חלופות בספירלה, benchmark, בדיקת היתכנות — פעולות שהיו יקרות ולכן דולגו, ונעשות עכשיו.

⚠️ ומה נעשה קשה יותר

המפרט חשוב יותר, לא פחות
ה-AI בונה בדיוק את מה שהגדרתם. אפיון רופף ← מערכת שגויה, מהר. תרחישי השימוש ותרשים המחלקות הם הקלט — ולכן ערכם עלה.

הצוואר עבר לבדיקה ולסקירה
אם הכתיבה מהירה פי חמישה והביקורת לא — הביקורת היא המגבלה. שם צריך להשקיע עכשיו: בדיקות אוטומטיות, סקירת קוד, הגדרת Done.

מהירות אינה התקדמות
קוד שנכתב אינו ערך. "תוכנה עובדת היא יחידת המידה" — העיקרון האג'ילי — נעשה חשוב פי כמה כשקל לייצר הרבה קוד שאיש לא בדק.

מה שלא זז בכלל
דרישות לא-פונקציונליות, אינטגרציה, אבטחה, תחזוקה ואחריות — עדיין של הצוות. וגם ההטמעה הארגונית: לשכנע את צוות המטבח.

אז מה עם משבר התוכנה?
הוא לא נגמר — הוא זז. המשבר מעולם לא היה על מהירות הכתיבה, אלא על מורכבות, תקשורת והבנת הצורך. ואת אלה ה-AI לא פותר. בהרצאה 13 נעשה בדיוק את זה בידיים: מפרט ← מערכת עובדת.

פיתוח

AI = Artificial Intelligence · בינה מלאכותית

סיכום

חמישה מודלים, ארבע שאלות — ואף מודל שמנצח תמיד

המודל	1 · גודל המנה	2 · הסדר	3 · הלקוח רואה	4 · שינוי
מפל מים Waterfall	כל המערכת	רצף, מעבר אחד	בסוף בלבד	הפרעה יקרה
אב-טיפוס Prototyping	מסך או תרחיש	לפני או בתוך מודל אחר	מוקדם מאוד	זו כל המטרה
תוספתי Incremental	תוספת מתפקדת	מעברים חוזרים	בסוף כל תוספת	לתוספת הבאה
ספירלה Spiral	לפי סיכון	סבבים, הכי מסוכן ראשון	אב-טיפוסים מוקדם	סיכון חדש ← סבב חדש
אג׳ייל Agile / Scrum	שבועיים עד חודש	חוזר, קצר וקבוע	בסוף כל ספרינט	זו התוכנית

1 • אין מנצח קבוע
לכל פרויקט בוחרים מתודולוגיה ומתאימים אותה. שילוב רעיונות מכמה מודלים הוא הנורמה, לא פשרה.

2 • יציבות הדרישות מכריעה
לא הגודל ולא התקציב. דרישות יציבות ← אפשר מפל מים. דרישות שיתבררו תוך כדי ← מנות קצרות.

3 • מה שהאיטרטיביות קונה
הקטנת סיכונים · תמיכה בשינויים · לימוד תוך כדי תנועה · שימוש חוזר · דרישות רלוונטיות · יכולת ניבוי טובה יותר.

4 • אג׳ייל ≠ בלי עיצוב
אותם תוצרים — גודל מנה אחר. ומודל שנבנה בפרוסות דורש דווקא יותר משמעת מידול.

5 • טקסים בלי ערכים לא עובדים
אפשר לאמץ ספרינטים ופגישות יומיות ולא לוותר על שליטה — ולקבל מפל מים עם פגישות.

6 • ה-AI מזיז את האופטימום
סבב זול ← מנה קטנה יותר משתלמת. אבל המפרט חשוב יותר, והצוואר עבר לבדיקה ולסקירה.

פיתוח

AI = Artificial Intelligence · בינה מלאכותית

מטלת בית

אותה מסעדה מהמטלה הקודמת — הפעם לא מה לבנות, אלא איך להריץ

התיק

'המלצר האוטומטי' החליטה על פתרון. עכשיו צריך להריץ את הפרויקט. מסמך האפיון המלא — מהרצאות 6–7.

- צוות: 3 מפתחים ואנליסט/ית במשרה חלקית
- יעד: עונת הקיץ, בעוד חמישה חודשים
- שלוש תחנות הכנה, ואף אחד לא בטוח איך הטבחים יגיבו למסך במקום פתקים
- המערכת נוגעת בכסף — תשלום וחלוקת חשבון
- בעל/ת המסעדה זמין/ה כל יום ורוצה להיות מעורב/ת

מה מגישים

1. בחרו מתודולוגיה ונמקו לפי ארבע השאלות — גודל מנה · סדר · מתי הלקוח רואה · איך מטפלים בשינוי. לכל שאלה, מה בתיק הוביל לתשובה
2. כתבו את שני הספרינטים הראשונים: מטרה לכל ספרינט, ופריטי בקלוג כסיפורי משתמש עם הערכה
3. הסבירו למה הספרינט הראשון הוא "הסקייטבורד" — מה הסועד כבר יכול לעשות בסופו
4. תנו ל-AI לטעון את ההפך — וכתבו תשובה נגדית של חצי עמוד

הפקודה ל-AI בסעיף 4

"בחרתי ב-[המתודולוגיה שלכם] לפרויקט הזה. טען בצורה המשכנעת ביותר למה זו הבחירה השגויה, ומה היה עדיף."

התוצר שנבדק הוא הנימוק והתשובה הנגדית. בחירת מתודולוגיה היא טיעון — ומי שלא יודע מה החולשות שלה, לא באמת בחר בה.

פיתוח

AI = Artificial Intelligence · בינה מלאכותית

תודה!

הבא בתור — מהמפרט למערכת עובדת: לוקחים את התוצרים של כל הסמסטר ובונים מהם מערכת, סבב אחד מקצה לקצה

ניתוח ועיצוב מערכות מידע · הרצאה 11 — מתודולוגיות פיתוח